our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Topaz Protocol

SECURITY REVIEW

Date: 25 April 2026

CONTENTS

1. About Shieldify Security **3**

2. Disclaimer **3**

3. About Topaz Protocol **3**

4. Risk classification **3**

4.1 Impact 3

4.2 Likelihood 4

5. Security Review Summary **4**

5.1 Protocol Summary 4

5.2 Scope 4

6. Findings Summary **5**

7. Findings **5**

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Topaz Protocol

Topaz is a ve(3,3)-style AMM and liquidity governance protocol on BNB Chain. It enables protocols, projects, and liquidity providers to coordinate sustainable, incentive-aligned liquidity through transparent vote-markets.

The ve(3,3) Solution: Instead of spraying emissions, the protocol lets token holders vote on where liquidity incentives flow, creating a competitive, market-driven system.

These repositories are a fork of Aerodrome's Slipstream, a concentrated liquidity implementation, adapted for use with Topaz DEX on BNB Chain. It contains the core concentrated liquidity contracts, adapted from UniswapV3's core contracts. It contains the higher-level periphery contracts, adapted from UniswapV3's periphery contracts. It also contains gauges designed to operate within the Topaz DEX ecosystem on BNB Chain.

Learn more about Topaz's contracts concept and the technicalities behind it: here

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable

- **Low** – losses will be limited but bearable – and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 5 days with a total of 80 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying one Medium and four Low severity issues. They're related to AMM / concentrated liquidity pool accounting, fee logic, and oracle manipulation issues in a Uniswap v3-style DEX implementation.

The Blue team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	Topaz Protocol
Repository	topazdex
Type of Project	AMM, DEX, Concentrated Liquidity, Uniswap V3 Fork
Security Review Timeline	5 days
Review Commit Hash #1	cd7c3ea18ee17cfd7642d37d9851728d8b7b1907
Review Commit Hash #2	f931a766bf3bad76b6217a0c41566013a7f167b9
Fixes Review Commit Hash #1	836f7f610499b6423d7dbad10c7822d51356b667
Fixes Review Commit Hash #2	1843a10faa3214e893578f27bae251b875fcc502

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
contracts/core/fees/DynamicSwapFeeModule.sol	149
contracts/core/interfaces/fees/IDynamicFeeModule.sol	12
Total	**161**

Chainges in topazdex/contracts and topazdex/slipstream/contracts

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: `1`
- **Low** issues: `4`
- **Info** issues: `2`

ID	Title	Severity	Status
[M-01]	Observation Cardinality Pre-Check Is Incorrect, Redundant and Based on Wrong Variable	Medium	Fixed
[L-01]	Discount Calculation Rounds Up in Favour of User Instead of Protocol	Low	Acknowledged
[L-02]	`MIN_SECONDS_AGO` Hardcoded to 2 Seconds, But BNB Chain Block Time Is 0.45 Seconds	Low	Acknowledged
[L-03]	`currentTick` Fetched from `slot0` Is Manipulable Via Spot Price Movement, Allowing Fee Griefing Against Targeted Swappers	Low	Acknowledged
[L-04]	`resetDynamicFee()` Does Not Reset `baseFee` Leaving Pool in Partially Reset State	Low	Acknowledged
[I-01]	Incorrect NatSpec Comment in `calculateGrowth()` Function	Info	Fixed
[I-02]	`MINIMUM_LIQUIDITY` Check in `mint()` Incorrectly Blocks Legitimate Small Depositors	Info	Acknowledged

7. Findings

[M-01] Observation Cardinality Pre-Check Is Incorrect, Redundant and Based on Wrong Variable

Severity

Medium Risk

Description

In `_getDynamicFee()`, a pre-check exists before calling `observe()` with the intention of skipping the external call when the pool does not have sufficient observation history:

```
if (observationCardinality < _secondsAgo / MIN_SECONDS_AGO) return 0;
```

Error 1 - Wrong variable used: `observationCardinality` represents the number of slots allocated in the circular observation buffer. It is set when someone calls `increaseObservationCardinalityNext()`

and has no relationship to how many observations have actually been written. The team likely intended to use `observationIndex`, which tracks how many observations have actually been written. But even that would not fix the fundamental problem described in Error 2.

Error 2 – Even the correct variable would not prevent `observe()` from reverting: `observe()` does not revert based on how many observations are written. It reverts only when the requested timestamp is older than the oldest stored observation in the buffer. Uniswap's oracle interpolates between available observations and can serve any timestamp within the available range, regardless of index value. Therefore, no simple comparison of index or cardinality against a threshold can reliably predict whether `observe()` will revert, only checking the actual oldest observation timestamp can do that.

Error 3 – The check is entirely redundant: The try/catch block immediately following already handles the `observe()` revert case correctly and returns 0. So, regardless of whether the pre-check passes or fails, the outcome is always to return 0 when there is insufficient history.

Location of Affected Code

File: contracts/core/fees/DynamicSwapFeeModule.sol#L177-L197

```
function _getDynamicFee(address _pool, uint256 _scalingFactor) internal
    view returns (uint256) {
  (, int24 currentTick, , uint16 observationCardinality, , ) = ICLPool(
      _pool).slot0();

  if (observationCardinality < _secondsAgo / MIN_SECONDS_AGO) return 0;

  try ICLPool(_pool).observe(sa) returns (int56[] memory tickCumulatives,
      uint160[] memory) {
      // code
  } catch {
      return 0;
  }
  // code
}
```

Impact

The developer confused `observationCardinality` (allocated slots) with observationIndex (written slots) and further assumed that a slot count comparison could predict `observe()` revert behavior. Neither assumption is correct.

Recommendation

Remove the pre-check entirely and rely solely on the try/catch block, which is already correct, chain-agnostic, and handles all failure cases.

Team Response

Fixed.

[L-01] Discount Calculation Rounds Up in Favour of User Instead of Protocol

Severity

Low Risk

Description

In the `getFee()` function, the discount amount is calculated using `FullMath.mulDivRoundingUp()`, which rounds the discount value upward. This means the user receives a slightly larger discount than they are entitled to, and the protocol collects a slightly smaller fee than intended.

Fee calculations in DeFi protocols should always round in favour of the protocol, not the user. This is a well-established convention across Uniswap, Aave, Compound and virtually every major protocol: when there is a fractional remainder in a fee or discount calculation, that remainder should stay with the protocol.

By using `mulDivRoundingUp()` for the discount, the code does the opposite, any fractional remainder is given to the user as an extra discount, reducing the fee collected below what the protocol is owed.

Location of Affected Code

File: contracts/core/fees/DynamicSwapFeeModule.sol#L170-L171

```
function getFee(address _pool) external view override returns (uint24) {
  // code
  uint256 discount = FullMath.mulDivRoundingUp(totalFee, discounted[tx.
      origin], 1_000_000);
  totalFee = totalFee - discount;
  // code
}
```

Impact

While individually negligible, this violates the protocol-favouring rounding invariant that all fee calculations should uphold.

Recommendation

Should round down instead of round up.

Team Response

Acknowledged.

[L-02] `MIN_SECONDS_AGO` Hardcoded to 2 Seconds, But BNB Chain Block Time Is 0.45 Seconds

Severity

Low Risk

Description

The contract hardcodes `MIN_SECONDS_AGO = 2` as the assumed block time for the target chain. This value is used in two critical places: the observation cardinality pre-check and the `MAX_SECONDS_AGO` upper bound calculation. BNB Chain's current block time is approximately 0.45 seconds following the BNB Chain speedup upgrade, which is roughly 4.4x faster than the hardcoded assumption.

This is not a configurable parameter. It is a compile-time constant that cannot be updated without redeploying the entire contract. The team has hardcoded a value that was already wrong for BNB at 3 seconds and is now dramatically wrong at 0.45 seconds.

Location of Affected Code

File: contracts/core/fees/DynamicSwapFeeModule.sol#L15-L16

```
uint32 public constant MIN_SECONDS_AGO = 2;
uint32 public constant MAX_SECONDS_AGO = 65535 * MIN_SECONDS_AGO;
```

Impact

Cardinality pre-check threshold massively inflated.

Recommendation

Change the value and make it according to the current block time. For BSC deployment:

```
-    uint32 public constant MIN_SECONDS_AGO = 2;
+    uint32 public constant MIN_SECONDS_AGO = 3; // BSC block time
```

Alternatively, make it configurable (bounded) rather than a constant:

```
-    uint32 public constant MIN_SECONDS_AGO = 2;
+    uint32 public MIN_SECONDS_AGO = 3;
+
+    function setMinSecondsAgo(uint32 _minSecondsAgo) external
   onlySwapFeeManager {
+        require(_minSecondsAgo >= 2 && _minSecondsAgo <= 15, 'ISA');
+        MIN_SECONDS_AGO = _minSecondsAgo;
+    }
```

Team Response

Acknowledged.

[L-03] `currentTick` Fetched from `slot0` Is Manipulable Via Spot Price Movement, Allowing Fee Griefing Against Targeted Swappers

Severity

Low Risk

Description

In `_getDynamicFee()`, the current tick is fetched directly from `slot0`. `slot0` returns the instantaneous spot price of the pool at the moment of the call. Unlike `twAvgTick`, which is averaged over 600 seconds and extremely resistant to manipulation, `currentTick` represents the price right now and can be moved by anyone who executes a sufficiently large swap in the same block or the block immediately preceding the victim's transaction.

The dynamic fee formula computes:

```
tickDelta    = currentTick - twAvgTick
absTickDelta = |tickDelta|
dynamicFee   = absTickDelta * scalingFactor / SCALING_PRECISION
```

Since `twAvgTick` is anchored to 600 seconds of price history, it moves very slowly. But `currentTick` can be pushed far away from `twAvgTick` in a single transaction by executing a large swap that moves the spot price. This artificially inflates absTickDelta and consequently inflates the dynamic fee charged to any swap that follows in the same block.

The attacker does not profit from this. The extra fee goes to LPs, not to the attacker. The attacker loses money executing the manipulative swap due to price impact and fees paid. This is a pure griefing attack, the goal is solely to cause the victim to pay a higher fee with no corresponding benefit to the attacker.

Location of Affected Code

File: contracts/core/fees/DynamicSwapFeeModule.sol#L177-L197

```
function _getDynamicFee(address _pool, uint256 _scalingFactor) internal
    view returns (uint256) {
  (, int24 currentTick, , uint16 observationCardinality, , ) = ICLPool(
      _pool).slot0();

  // code

  int24 tickDelta = currentTick - twAvgTick;
  uint24 absTickDelta = tickDelta < 0 ? uint24(-tickDelta) : uint24(
      tickDelta);
  return (absTickDelta * _scalingFactor) / SCALING_PRECISION;
}
```

Impact

Users are experiencing unexpected high fees with no market movement visible on price charts.

Recommendation

Instead of comparing spot price to long TWAP, compare short TWAP to long TWAP. Both sides are now averaged and manipulation-resistant.

Team Response

Acknowledged.

[L-04] `resetDynamicFee()` Does Not Reset `baseFee` Leaving Pool in Partially Reset State

Severity

Low Risk

Description

The `resetDynamicFee()` function is intended to reset the dynamic fee configuration for a pool. It deletes `feeCap` and `scalingFactor` from the `DynamicFeeConfig` struct but leaves `baseFee` untouched. All three fields are part of the same configuration unit.

A fee manager calling `resetDynamicFee()` reasonably expects the pool to be fully returned to its default state, meaning `baseFee` should also be cleared so the pool falls back to `factory.tickSpacingToFee(tickSpacing)` as its base. Instead, the custom `baseFee` silently persists after the reset, leaving the pool in a hybrid state that is neither fully custom nor fully default.

Location of Affected Code

File: contracts/core/fees/DynamicSwapFeeModule.sol#L116-L122

```
function resetDynamicFee(address _pool) external override
    onlySwapFeeManager {
     require(factory.isPool(_pool));
     delete dynamicFeeConfig[_pool].feeCap;
     delete dynamicFeeConfig[_pool].scalingFactor;
     // delete dynamicFeeConfig[_pool].baseFee   missing
     emit DynamicFeeReset({pool: _pool});
}
```

Impact

Pool uses stale custom baseFee after reset instead of factory default fee tier.

Recommendation

Delete the base fee as well.

```
function resetDynamicFee(address _pool) external override
    onlySwapFeeManager {
      require(factory.isPool(_pool));

+     delete dynamicFeeConfig[_pool].baseFee;
      delete dynamicFeeConfig[_pool].feeCap;
      delete dynamicFeeConfig[_pool].scalingFactor;
```

```
        emit DynamicFeeReset({pool: _pool});
    }
```

Team Response

Acknowledged.

[I-01] Incorrect NatSpec Comment in `calculateGrowth()` Function

Severity

Informational Risk

Description

The NatSpec comment describing the rebase formula in `calculateGrowth()` is incorrect and does not match the actual implementation. The comment still documents the old Aerodrome/Velodrome formula, which used the unlocked share of supply (total minus locked), squared, divided by two.

The actual implementation was changed to use the locked share of supply (`veTotal` directly), cubed, divided by two. Two distinct errors exist in the comment simultaneously: the wrong ratio is described (subtraction-based unlocked share instead of direct locked share), and the wrong exponent is stated (squared instead of cubed).

Location of Affected Code

File: contracts/interfaces/IMinter.sol#L133-L139

```
/// @notice Calculates rebases according to the formula
///         weekly * ((topaz.totalsupply - ve.totalSupply) / topaz.
    totalsupply) ^ 2 / 2
///         Note that ve.totalSupply is the locked ve supply
///         topaz.totalSupply is the total ve supply minted
```

Recommendation

Update the NatSpec comment in both `IMinter.sol` and `Minter.sol` to accurately reflect the actual formula being computed.

Team Response

Fixed.

[I-02] `MINIMUM_LIQUIDITY` Check in `mint()` Incorrectly Blocks Legitimate Small Depositors

Severity

Informational Risk

Description

The `mint()` function enforces a check that reverts the transaction if the computed liquidity tokens to be issued are less than `MINIMUM_LIQUIDITY`. While this check was introduced to prevent dust liquidity attacks, it has an unintended side effect of blocking entirely legitimate small depositors who have no malicious intent.

Any user who provides a deposit small enough that their resulting LP token amount falls below `MINIMUM_LIQUIDITY` will have their transaction reverted with `InsufficientLiquidityMinted`, even though they are depositing genuine proportional liquidity into the pool. The check treats all small deposits as suspicious, regardless of whether they pose any actual attack risk.

Location of Affected Code

File: contracts/Pool.sol#L305-L328

```
function mint(address to) external nonReentrant returns (uint256
    liquidity) {
    (uint256 _reserve0, uint256 _reserve1) = (reserve0, reserve1);
    uint256 _balance0 = IERC20(token0).balanceOf(address(this));
    uint256 _balance1 = IERC20(token1).balanceOf(address(this));
    uint256 _amount0 = _balance0 - _reserve0;
    uint256 _amount1 = _balance1 - _reserve1;
    uint256 _totalSupply = totalSupply();

    if (_totalSupply == 0) {
        liquidity = Math.sqrt(_amount0 * _amount1) - MINIMUM_LIQUIDITY;
        _mint(address(1), MINIMUM_LIQUIDITY);
    } else {
        liquidity = Math.min(
            (_amount0 * _totalSupply) / _reserve0,
            (_amount1 * _totalSupply) / _reserve1
        );
    }

    if (liquidity < MINIMUM_LIQUIDITY) revert InsufficientLiquidityMinted
        (); //  problematic check
    _mint(to, liquidity);
}
```

Impact

This check creates a hidden minimum deposit size and it is not obvious to users or integrators.

Recommendation

Remove the check and only put a greater-than-zero check for liquidity.

File: contracts/UniswapV2Pair.sol#L125

```
require(liquidity > 0, 'UniswapV2: INSUFFICIENT_LIQUIDITY_MINTED');
```

Team Response

Acknowledged.

our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Thank you!